Operadores en Python 🌟

Aprendimos que las `variables` nos permiten almacenar diferentes tipos de datos, funcionando como contenedores de información. Sin embargo, no basta con solo guardar estos valores, nuestra verdadera misión es interactuar con ellos y transformarlos en cada paso de nuestros programas.

Para lograr esta transformación, vamos a utilizar los `operadores`. En esta sección, aplicaremos operaciones matemáticas que ya conocemos y descubriremos algunas herramientas nuevas y sumamente potentes que `Python` nos ofrece para manipular estos datos.

Operaciones aritméticas

Suma `+`

El primero de estos operadores es la suma. En `Python`, utilizamos el símbolo `+` para combinar valores. Nos sirve, por ejemplo, para acumular puntos o calcular totales.

```python
income = 400
extras = 65

# Sumamos ambos valores para obtener el total
total_sum = income + extras

# Mostramos el resultado
print(total_sum)
```

Copiar

✓ 465

Resta `-`

Para encontrar la diferencia entre dos números, utilizamos el operador `-`.

```python
initial_budget = 1000
total_expenses = 450

# Restamos el valor de la variable 2 a la variable 1
remaining_balance = initial_budget - total_expenses

# Mostramos el valor restante
print(remaining_balance)
```

Copiar

✓ 550

Multiplicación `*`

Para realizar la multiplicación de dos factores utilizamos el carácter asterisco `*`.

```
1  unit_price = 20
2  product_quantity = 5
3
4  # Calculamos el total multiplicando
5  total_cost = product_quantity * unit_price
6
7  # Mostramos el costo final
8  print(total_cost)
```

Copiar

✓ 100

División `/`

Cuando usamos el operador `/`, realizamos una división. Es importante recordar que en `Python`, este operador siempre nos devolverá un número decimal (un `float`), **incluso** si el resultado es exacto.

```python
pizza_slices = 25
party_members = 4

# Calculamos los slices por persona
slices_per_member = pizza_slices / party_members

# Mostramos el resultado
print(slices_per_member)
```

Copiar

✓ 6.25

En algunas ocasiones, necesitamos hacer una división ignorando los decimales, donde queremos el cociente **entero**. Para esto, utilizamos el operador de división entera `//` .

```python
pizza_slices = 25
party_members = 4

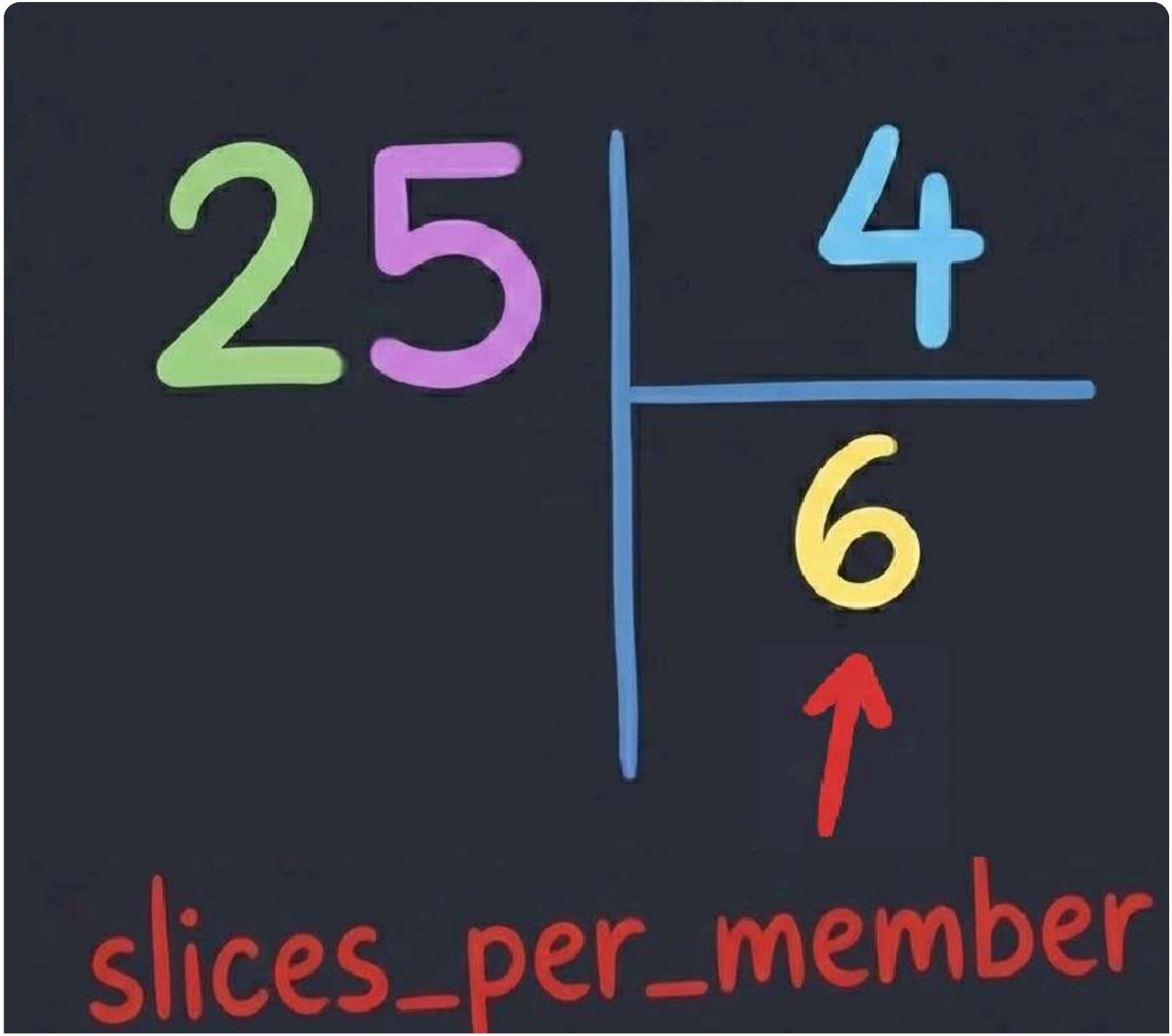
# Calculamos los slices enteros por persona
slices_per_member = pizza_slices // party_members

# Mostramos el resultado
print(slices_per_member)
```

Copiar

✓ 6

El ejercicio anterior obtenemos como resultado `6` ya que al hacer la división, el cociente entero de realizar la división de `25` entre `4`, es `6`. Tomar en cuenta que esto **no es** un redondeo. Si el resultado es `1.99`, terminará como `1`.


25
4
6
slices_per_member

Módulo `%`

El operador `%` nos permite obtener el "*sobrante*" de una división entera. Es extremadamente útil, por ejemplo para determinar si un número es par o impar o para conversión de unidades.

```
1  total_items = 17
2  box_capacity = 3
3
4  # Obtenemos lo que sobra después de llenar las cajas
5  leftover_items = total_items % box_capacity
6
7  # Mostramos el residuo de la operación
8  print(leftover_items)
```

Copiar

✓ 2

El ejercicio anterior obtenemos como resultado `2` ya que al hacer la división, el residuo de realizar la división de `17` entre `3`, es `2`.



Potencia ^

Para realizar cálculos exponenciales, como elevar un número al cuadrado o al cubo, utilizamos el doble asterisco `**`.

```python
base_number = 2
exponent_value = 3

# Elevamos la base al exponente (2 al cubo) 2 * 2 * 2
calculation_result = base_number ** exponent_value

# Mostramos el resultado de la potencia
print(calculation_result)
```

Copiar

✓ 8

Resumen

Operación	Símbolo	Descripción	Ejemplos
Suma	`+`	Suma dos números	`10 + 35 → 45`
Resta	`-`	Resta un numero de otro	`90 - 50 → 40`
Multiplicación	`*`	Multiplica dos números	`3 * 5 → 15`
Division	`/`	Divide un numero por otro	`10 / 3 → 3.33`
Division entera	`//`	Divide un numero por otro pero quitando los decimales del resultado	`10 // 3 → 3`
Potencia	`**`	Eleva un numero a la potencia de otro	`2 ** 3 → 8 (2 * 2 * 2)`

Operación	Símbolo	Descripción	Ejemplos
Mod	`%`	Divide un numero por otro pero retorna su residuo (como el que vimos en Pseudocódigo)	`10 % 3 → 1`

Operaciones con strings ✍🏻

Antes de concluir con la lección, hay que mencionar que hay funcionalidades dentro de `Python` como "*sumar*" `strings`, lo cual es súper importante que conozcamos. A este proceso de unir dos o más `strings` lo llamamos **concatenación** y lo podemos conseguir de múltiples maneras.

Concatenación de Strings +

La primera forma de lograrlo es mediante el operador de suma. Para ello, unimos múltiples `strings` y los unimos en un `string` resultante. A esto le llamamos concatenación.

Copiar

```python
first_word = "Hola"
second_word = "Mundo"

# Unimos las palabras y un espacio en blanco con el símbolo +
full_greeting = first_word + " " + second_word

# Mostramos el texto unido
print(full_greeting)
```

✓ Hola Mundo

Format Strings `f-string` 🟩🟦

La segunda forma es mediante `f-strings`, los cuales son la forma más moderna y elegante de unir variables dentro de un texto. Para crear un `f-string` solo necesitamos anteponer la letra `f` al inicio de las comillas, y agregar las variables dentro de llaves `{}`.

Copiar

```
1  user_name = "Andrés"
2  user_age = 26
3
4  # Creamos el mensaje usando f-strings para unir los valores
5  welcome_message = f"Hola {user_name}, tu edad es {user_age} años."
6
7  # Mostramos el mensaje formateado
8  print(welcome_message)
```

✓ Hola Andrés, tu edad es 26 años.

Reemplazo de llaves `string.format()` 🔑

Este es el método tradicional de unir `strings`. Utilizamos llaves `{}` como espacios reservados que luego serán rellenados por los valores que pasemos dentro de `.format()`.

Copiar

```python
item_name = "PC Gamer"
item_price = 800

# Formateamos el texto con el método format
price_tag = "El {} tiene un costo de {} USD.".format(item_name, item_price)

# Mostramos el mensaje final
print(price_tag)
```

El PC Gamer tiene un costo de 800 USD.
